Patrones de Diseño en mi proyecto: Simulador de Combates de Dragon Ball

Diego

Mayo 5, 2026

Índice

1. Introducción **2**

2. Patrones Creacionales **2**
2.1. Singleton 2
2.2. Factory Method 3
2.3. Builder 6

3. Patrones Estructurales **8**
3.1. Repository 8
3.2. Adapter 10
3.3. Facade 11

4. Conclusión **13**

1. Introducción

Los patrones de diseño son soluciones probadas a problemas comunes en el desarrollo de software. Se dividen en tres grupos principales: los **creacionales**, que tienen que ver con cómo se crean los objetos; los **estructurales**, que hablan de cómo se organizan entre sí, y los **de comportamiento**, que definen cómo interactúan y se comunican.

En este documento explico los patrones que utilicé en mi proyecto, *Simulador de peleas de Dragon Ball*, el cual está compuesto por:

- **Backend:** API REST en Java con Spring Boot 3, Spring Security y JWT, conectada a una base de datos PostgreSQL en Supabase y desplegada en Render.
- **Frontend:** Aplicación móvil en Flutter (Dart) que consume la API y sube imágenes directamente a Cloudinary.

2. Patrones Creacionales

Estos patrones se encargan de controlar cómo se crean los objetos dentro de una aplicación. Puede parecer que eso es lo más simple del mundo (solo haces `new Objeto()` y listo), pero en proyectos más grandes eso puede volverse un problema.

2.1. Singleton

La idea es que de una clase solo exista una única instancia en todo el programa. Imagínate que tienes una clase que maneja la conexión a la base de datos: no tiene sentido crear cien conexiones distintas, con una sola alcanza.

En el proyecto utilicé el Singleton para el `AuthService` en Flutter, donde guardo la sesión del usuario. No quería que hubiera varios objetos manejando el token al mismo tiempo porque eso generaría inconsistencias.

Implementación en el Frontend (Dart / Flutter)

La clase `AuthService` declara un campo estático `instance` que es la única instancia posible. El constructor privado `AuthService._()` impide que cualquier otra parte del código pueda crear un objeto nuevo.

```
class AuthService {
  static const _keyToken   = 'auth_token';
  static const _keyUsuario = 'auth_usuario';

  // Unica instancia global; se accede con AuthService.
      instance
  static final AuthService instance = AuthService._();
```

```dart
  AuthService._();   // Constructor privado: nadie mas puede instanciar

  Usuario? _usuario;

  Usuario? get usuario       => _usuario;
  bool     get estaAutenticado => _usuario != null;
  String?  get token           => _usuario?.token;

  // Cargar la sesion guardada al arrancar la app
  Future<void> cargarSesion() async {
    final prefs = await SharedPreferences.getInstance();
    final json  = prefs.getString(_keyUsuario);
    if (json != null) {
      try {
        _usuario = Usuario.fromJson(jsonDecode(json));
      } catch (_) {
        await cerrarSesion();
      }
    }
  }
  // ... resto de metodos (login, registrar, cerrarSesion)
}
```

Listing 1: AuthService – patrón Singleton (lib/services/auth_service.dart)

En `main.dart` se accede al singleton antes de arrancar la app:

```dart
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  // Se usa la instancia unica para cargar el token guardado
  await AuthService.instance.cargarSesion();
  runApp(const DragonBallApp());
}
```

Listing 2: Uso del Singleton en main.dart

Cualquier widget del proyecto puede obtener el token actual con `AuthService.instance.token` sin necesidad de pasar ningún objeto entre pantallas.

La desventaja que le veo es que a veces complica las pruebas, porque el estado global puede interferir entre tests, pero en general es muy útil.

2.2. Factory Method

Este patrón básicamente dice: en lugar de crear un objeto directamente, delegas esa responsabilidad a un método o clase especial que decide qué tipo de objeto crear.

Lo usé de forma informal cuando el backend decide qué tipo de respuesta construir dependiendo del resultado de la operación. Además, en Dart los constructores nombrados `fromJson` son una variante directa del Factory Method: delegan la creación del objeto a un método que sabe cómo interpretar los datos entrantes.

Implementación en el Frontend (Dart / Flutter)

Los modelos `Combate` y `Usuario` usan constructores `factory` que actúan como fábrica: reciben un `Map` JSON y devuelven una instancia correctamente inicializada.

```dart
class Combate {
  final int?   id;
  final int    anio;
  final String intensidad;
  final String tipo;
  final String lugar;
  final String creador;
  final int?   usuarioId;
  final String? resultado;
  final String? fechaCreacion;
  final ImagenCombate? imagen;

  Combate({ this.id, required this.anio, required this.intensidad,
            required this.tipo, required this.lugar, required this.creador,
            this.usuarioId, this.resultado, this.fechaCreacion, this.imagen });

  // Factory Method: decide como construir el objeto segun el JSON recibido
  factory Combate.fromJson(Map<String, dynamic> j) => Combate(
    id:           j['id'],
    anio:         j['anio'],
    intensidad:   j['intensidad'],
    tipo:         j['tipo'],
    lugar:        j['lugar'],
    creador:      j['creador'],
    usuarioId:    j['usuarioId'],
    resultado:    j['resultado'],
    fechaCreacion: j['fechaCreacion'],
    imagen: j['imagen'] != null
        ? ImagenCombate.fromJson(j['imagen'])
        : null,   // <-- decision segun el dato disponible
  );
}
```

Listing 3: Factory Method en el modelo Combate (lib/models/combate.dart)

Implementación en el Backend (Java / Spring Boot)

En el backend, `AuthService` construye distintos tipos de respuesta según la operación (registro o login), encapsulando la lógica de creación del objeto `AuthDtos.AuthResponse`.

```java
@Service
public class AuthService {

    private final UsuarioRepository repo;
    private final PasswordEncoder   encoder;
    private final JwtUtil           jwtUtil;

    public AuthDtos.AuthResponse registrar(AuthDtos.RegisterRequest req) {
        if (repo.findByEmail(req.getEmail()).isPresent())
            throw new IllegalArgumentException("Email ya registrado");

        Usuario u = new Usuario();
        u.setNombre(req.getNombre());
        u.setEmail(req.getEmail());
        u.setPassword(encoder.encode(req.getPassword()));
        repo.save(u);

        String token = jwtUtil.generarToken(u.getEmail());
        // Factory: construye la respuesta segun el contexto (registro)
        return new AuthDtos.AuthResponse(
                token, u.getId(), u.getNombre(), u.getEmail());
    }

    public AuthDtos.AuthResponse login(AuthDtos.LoginRequest req) {
        Usuario u = repo.findByEmail(req.getEmail())
                .orElseThrow(() -> new IllegalArgumentException(
                        "Usuario no encontrado"));
        if (!encoder.matches(req.getPassword(), u.getPassword()))
            throw new IllegalArgumentException("Contrasena incorrecta");

        String token = jwtUtil.generarToken(u.getEmail());
        // Factory: construye la respuesta segun el contexto (login)
```

```java
        return new AuthDtos.AuthResponse(
                token, u.getId(), u.getNombre(), u.getEmail
                    ());
    }
}
```

Listing 4: Creación de respuesta en AuthService (service/AuthService.java)

2.3. Builder

El Builder sirve cuando tienes objetos que se construyen paso a paso y que tienen muchos campos opcionales. En vez de tener un constructor con veinte parámetros (que nadie entiende en qué orden van), usas un objeto intermedio que vas configurando.

En mi proyecto, cuando se construye un objeto `Combate` con lugar, año, intensidad, tipo, imagen y resultado, se podría usar un Builder para ir armándolo sin confundirse con los parámetros. Yo no lo implementé como patrón formal, pero la misma idea está presente en el formulario de creación de combates en Flutter: el usuario llena campo por campo antes de mandar todo al backend, y en el backend, `CombateService` también construye el resultado del combate de forma incremental.

Implementación en el Backend (Java / Spring Boot)

El método `simularCombate` es el mejor ejemplo: comienza con un `Combate` base y lo va enriqueciendo paso a paso (cálculo de stats, bonus por intensidad, bonus por tipo, bonus por lugar, evento aleatorio, ganador final) antes de persistirlo.

```java
public Combate simularCombate(Long id) {
    Optional<Combate> opt = repo.findById(id);
    if (opt.isEmpty()) return null;
    Combate c = opt.get();

    // Paso 1: stats base
    int ataque    = random.nextInt(100) + 1;
    int defensa   = random.nextInt(100) + 1;
    int velocidad = random.nextInt(100) + 1;

    // Paso 2: bonus segun intensidad
    switch (c.getIntensidad().toLowerCase()) {
        case "epico"  -> { ataque += 20; velocidad += 15; }
        case "alto"   -> { ataque += 10; velocidad += 5;  }
        case "medio"  ->   ataque += 5;
    }

    // Paso 3: bonus segun tipo de combate
    switch (c.getTipo().toLowerCase()) {
        case "supervivencia" -> defensa += 15;
```

```java
        case "equipo"        -> { ataque += 10; defensa += 10; }
    }

    // Paso 4: bonus segun lugar
    switch (c.getLugar().toLowerCase()) {
        case "namek"  -> ataque    += 10;
        case "torneo" -> velocidad += 10;
        case "tierra" -> defensa   += 5;
    }

    // Paso 5: evento y ganador
    String evento  = generarEvento();
    String ganador = (ataque + velocidad) >= (defensa + random.nextInt(80))
            ? "Guerrero A" : "Guerrero B";

    // Paso 6: ensamblar el resultado final y persistir
    c.setResultado(
        "Lugar: "      + c.getLugar()      +
        " | Intensidad: " + c.getIntensidad() +
        " | Evento: "  + evento            +
        " | ATQ:" + ataque + " DEF:" + defensa + " VEL:" + velocidad +
        " | Ganador: " + ganador
    );
    return repo.save(c);
}
```

Listing 5: Construcción incremental del resultado en CombateService (service/CombateService.java)

Reflejo en el Frontend (Dart / Flutter)

En la pantalla `CrearCombateScreen` el usuario va llenando campo por campo (año, intensidad, tipo, lugar, creador, imagen), y solo cuando todo está listo se ensambla el objeto y se envía:

```dart
// Cada campo se guarda en su variable de estado
String _lugar      = '';
String _intensidad = 'Medio';
String _tipo       = 'Individual';
int    _anio       = 2024;
String _creador    = '';

// Al confirmar, se ensambla el objeto completo
void _guardar() async {
  final combate = Combate(
    anio:       _anio,
    intensidad: _intensidad,
```

```
    tipo:       _tipo,
    lugar:      _lugar,
    creador:    _creador,
  );
  await _repo.crearCombate(combate);  // Se envia solo
      cuando esta completo
}
```

Listing 6: Construcción incremental del Combate en el formulario (screens/crear_combate_screen.dart – fragmento)

3. Patrones Estructurales

Estos patrones se enfocan en cómo se relacionan y se organizan las clases entre sí. Sirven para que el código sea más flexible y fácil de mantener sin tener que cambiar todo cada vez que algo cambia.

3.1. Repository

Este fue el que más usé en el proyecto. La idea es poner una capa entre la lógica de la aplicación y el origen de los datos (que puede ser una base de datos, una API, lo que sea).

En Flutter tengo una clase `CombateRepository` que es la única que sabe cómo hablar con el backend. Las pantallas no llaman directamente al servicio HTTP, sino que le piden al repositorio. Así, si en algún momento cambia la URL o la forma de obtener datos, solo tengo que modificar el repositorio y no tocar todas las pantallas.

Implementación en el Frontend (Dart / Flutter)

```
import '../models/combate.dart';
import '../services/api_service.dart';

class CombateRepository {
  final _api = ApiService();   // Dependencia interna oculta
      al caller

  // Las pantallas solo conocen estos metodos, no saben nada
      de HTTP
  Future<List<Combate>> obtenerCombates()       => _api.
      obtenerCombates();
  Future<List<Combate>> obtenerMisCombates()    => _api.
      obtenerMisCombates();
  Future<Combate>       obtenerPorId(int id)    => _api.
      obtenerPorId(id);
  Future<Combate>       crearCombate(Combate c) => _api.
      crearCombate(c);
```

```
  Future<Combate>       actualizarCombate(int id, Combate c)
                                                => _api.
                                                    actualizarCombate
                                                    (id, c);
  Future<Combate>       actualizarImagen(int id, String url)
                                                => _api.
                                                    actualizarImagen
                                                    (id, url)
                                                    ;
  Future<void>          eliminarCombate(int id) => _api.
      eliminarCombate(id);
  Future<Combate>       simularCombate(int id)  => _api.
      simularCombate(id);

  Future<Map<String, int>>     obtenerReacciones(int id)
                                                => _api.
                                                    obtenerReacciones
                                                    (id);
  Future<List<String>>         obtenerMisReacciones(int id)
                                                => _api.
                                                    obtenerMisReacciones
                                                    (id);
  Future<Map<String, dynamic>> toggleReaccion(int id, String
       t)
                                                => _api.
                                                    toggleReaccion
                                                    (id, t);
}
```

Listing 7: CombateRepository – capa de acceso a datos (lib/repositories/combate_repository.dart)

Implementación en el Backend (Java / Spring Boot)

En el backend, Spring Data JPA aplica el mismo patrón de forma automática. `CombateRepository` extiende `JpaRepository` y define consultas personalizadas sin que el servicio sepa nada de SQL:

```
package com.example.dragon.repository;

import com.example.dragon.entity.Combate;
import org.springframework.data.jpa.repository.JpaRepository
    ;
import java.util.List;

public interface CombateRepository extends JpaRepository<
    Combate, Long> {
    // Spring genera automaticamente el SQL; el servicio
        solo llama al metodo
    List<Combate> findByUsuarioId(Long usuarioId);
```

```
}
```

Listing 8: CombateRepository en Spring Data JPA (repository/CombateRepository.java)

3.2. Adapter

El Adapter sirve cuando tienes dos partes del sistema que no hablan el mismo idioma y necesitas un intermediario que traduzca.

En mi proyecto lo vi reflejado en los métodos `fromJson()` de los modelos en Dart: el backend devuelve JSON con ciertos nombres de campos, y el modelo se encarga de traducirlos al formato que usa Flutter internamente. De forma equivalente, `toJson()` hace la traducción en sentido contrario.

Implementación en el Frontend (Dart / Flutter)

```dart
class Combate {
  // ...campos internos de Flutter...

  // Adapter: convierte JSON del backend -> objeto Dart
  factory Combate.fromJson(Map<String, dynamic> j) => Combate(
    id:           j['id'],
    anio:         j['anio'],
    intensidad:   j['intensidad'],
    tipo:         j['tipo'],
    lugar:        j['lugar'],
    creador:      j['creador'],
    usuarioId:    j['usuarioId'],    // clave snake_case del backend
    resultado:    j['resultado'],
    fechaCreacion: j['fechaCreacion'],
    imagen: j['imagen'] != null
        ? ImagenCombate.fromJson(j['imagen'])
        : null,
  );

  // Adapter inverso: convierte objeto Dart -> JSON para el backend
  Map<String, dynamic> toJson() => {
    'anio':       anio,
    'intensidad': intensidad,
    'tipo':       tipo,
    'lugar':      lugar,
    'creador':    creador,
  };
}
```

Listing 9: Adapter JSON↔Dart en el modelo Combate (lib/models/combate.dart)

Implementación en el Backend (Java / Spring Boot)

En el backend, los DTOs de `AuthDtos` cumplen el mismo rol: adaptan los datos que llegan del cliente al formato que usa la lógica interna, y viceversa para las respuestas.

```java
public class AuthDtos {

    // Adapter de entrada: traduce el JSON del cliente a un objeto Java
    public static class LoginRequest {
        private String email;
        private String password;
        public String getEmail()    { return email; }
        public String getPassword() { return password; }
    }

    // Adapter de salida: traduce el objeto Java al JSON que espera Flutter
    public static class AuthResponse {
        private final String token;
        private final Long   usuarioId;   // nombre que usa Flutter
        private final String nombre;
        private final String email;

        public AuthResponse(String token, Long usuarioId,
                            String nombre, String email) {
            this.token     = token;
            this.usuarioId = usuarioId;
            this.nombre    = nombre;
            this.email     = email;
        }
        // getters...
    }
}
```

Listing 10: Adapter entre capas con DTOs (dto/AuthDtos.java – fragmento)

3.3. Facade

El Facade es básicamente crear una clase sencilla que oculte la complejidad interna. Desde afuera ves una interfaz simple, pero por dentro puede estar haciendo muchas cosas.

Mi `ApiService` del proyecto funciona exactamente así: las pantallas solo llaman métodos como `obtenerCombates()` sin preocuparse por los detalles HTTP, el manejo del token, la serialización JSON, etc. Todo eso está escondido dentro del `ApiService`.

Implementación en el Frontend (Dart / Flutter)

```dart
class ApiService {
  static const String baseUrl = 'https://dragonw.onrender.com';
  static const String _path   = '$baseUrl/api/combates';
  static const String _rpath  = '$baseUrl/api/reacciones';

  // Detalle interno: headers publicos
  static const Map<String, String> _headersPublic = {
    'Content-Type': 'application/json',
  };

  // Detalle interno: headers con JWT inyectado automaticamente
  Map<String, String> get _headers {
    final token = AuthService.instance.token;
    return {
      'Content-Type': 'application/json',
      if (token != null) 'Authorization': 'Bearer $token',
    };
  }

  // Interfaz simple que usan las pantallas: un solo metodo, cero detalles HTTP
  Future<List<Combate>> obtenerCombates() async {
    final res = await http.get(Uri.parse(_path));
    if (res.statusCode == 200) {
      final list = jsonDecode(res.body) as List;
      return list.map((e) => Combate.fromJson(e)).toList();
    }
    throw Exception('Error ${res.statusCode} al obtener combates');
  }

  // Toggle de reaccion: internamente hace POST, maneja token, decodifica JSON
  Future<Map<String, dynamic>> toggleReaccion(
      int combateId, String tipo) async {
    final res = await http.post(
      Uri.parse('$_rpath/$combateId/toggle'),
      headers: _headers,                        // JWT oculto al caller
      body: jsonEncode({'tipo': tipo}),
    );
```

```dart
    if (res.statusCode == 200) {
      final data   = jsonDecode(res.body) as Map<String, dynamic>;
      final conteo = data['conteo'] as Map<String, dynamic>;
      return {
        'added': data['added'] as bool,
        'tipo':  data['tipo']  as String,
        'like':  (conteo['like']  as num).toInt(),
        'heart': (conteo['heart'] as num).toInt(),
        'laugh': (conteo['laugh'] as num).toInt(),
      };
    }
    throw Exception('Error ${res.statusCode}: ${res.body}');
  }
  // ... todos los demas metodos siguen el mismo patron
}
```

Listing 11: ApiService – Facade sobre HTTP (lib/services/api_service.dart)

Gracias al Facade, el código de las pantallas queda limpio y enfocado solo en mostrar información:

```dart
// La pantalla no sabe nada de HTTP, tokens ni JSON
final combates = await _repo.obtenerCombates();
setState(() => _combates = combates);
```

Listing 12: Uso del Facade desde una pantalla (fragmento de home_screen.dart)

4. Conclusión

Al principio me parecía que los patrones de diseño son muy difíciles de aplicar en proyectos reales, pero conforme fui entendiendo y practicando en clase, y al mismo tiempo aplicándolos en mi proyecto, me di cuenta de que los estaba usando sin saberlo, o que muchas cosas que hacía por intuición tenían un nombre y una razón de ser.

No creo que haya que memorizar todos los patrones ni intentar usarlos todos en un mismo proyecto. La clave es entender el problema que resuelve cada uno y tenerlos en mente para cuando uno los necesite.